МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 4

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23 Враженко Д. О.

_____
(подпись студента)

Принял преподаватель Тесла-Блозва С. А.

_____
(подпись руководителя)

Работа представлена «____»_____ 2026 г.

Допущен к работе «___» _____ 2026 г.

Москва 2026

-

Оглавление

1. Цель работы.....3
2. Ход выполнения работы.....3
    2.1 Часть 1. Создайте сущность справочника.....3
    2.2 Часть 2. Реализуйте интерфейс управления справочниками.....4
    2.3 Часть 3. Интегрируйте справочник в форму документа.....6
    2.4 Часть 4. Стилизация и кастомизация (UI Tuning).....11
3. Вывод.....17
4. Ответы на контрольные вопросы.....18

1. Цель работы

Создать справочник «Контрагенты», связать его с основной сущностью документа и выполнить стилизацию интерфейса под стандарты платформы.

2. Ход выполнения работы

2.1 Часть 1. Создайте сущность справочника

Содержание класса `Counterparty.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import jakarta.persistence.*;

@Entity
@Table(name = "counterparties")
public class Counterparty {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "inn")
    private String inn;

    @Column(name = "address")
    private String address;

    public Counterparty() {}

    public Counterparty(String name, String inn, String address)
{
        this.name = name;
        this.inn = inn;
        this.address = address;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getInn() { return inn; }
    public void setInn(String inn) { this.inn = inn; }
```

```
    public String getAddress() { return address; }
      public void setAddress(String address) { this.address =
address; }
}
```

Содержание интерфейса CounterpartyRepository.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import org.springframework.data.jpa.repository.JpaRepository;

public interface CounterpartyRepository extends
JpaRepository<Counterparty, Long> {}
```

Содержание класса CounterpartyService.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import org.springframework.data.jpa.repository.JpaRepository;

public interface CounterpartyRepository extends
JpaRepository<Counterparty, Long> {}
```

Результат добавления нового поля в класс Documet.java представлен на Рисунке 1.

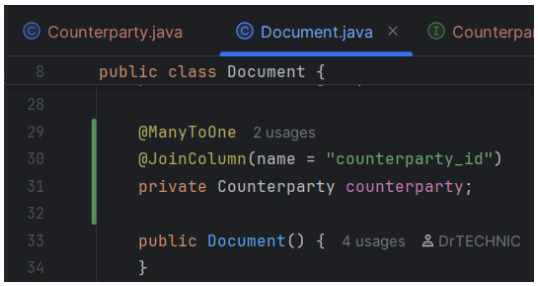


Рисунок 1 – Результат нового поля

2.2 Часть 2. Реализуйте интерфейс управления справочниками

Содержание класса CounterpartyView.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.button.ButtonVariant;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.grid.Grid;
```

```
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Counterparty;
import com.vrazhenko.documentfeature.CounterpartyService;
import org.springframework.beans.factory.annotation.Autowired;

@Route(value = "counterparties", layout = MainLayout.class)
public class CounterpartyView extends VerticalLayout {

    private final CounterpartyService service;
            private   final   Grid<Counterparty>   grid   =   new
Grid<>(Counterparty.class);

                private    final    TextField    name    =    new
TextField("Наименование");
    private final TextField inn = new TextField("ИНН");
    private final TextField address = new TextField("Адрес");

          private   final   Binder<Counterparty>   binder   =   new
Binder<>(Counterparty.class);
           private   Counterparty   currentCounterparty   =   new
Counterparty();

        public  CounterpartyView(@Autowired  CounterpartyService
service) {
        this.service = service;
        grid.setColumns("name", "inn", "address");
        grid.getColumnByKey("name").setHeader("name");
        grid.getColumnByKey("inn").setHeader("INN");
        grid.getColumnByKey("address").setHeader("address");
                                grid.getColumns().forEach(col    ->
col.setAutoWidth(true));
        updateGrid();
            FormLayout  formLayout  =  new  FormLayout(name,  inn,
address);
        Button saveButton = new Button("Сохранить");
         saveButton.addThemeVariants(ButtonVariant.LUMO_PRIMARY)
;
        saveButton.addClickListener(event -> {
            if (binder.writeBeanIfValid(currentCounterparty)) {
                service.save(currentCounterparty);
                updateGrid();
                clearForm();
            }
        });
               binder.forField(name).bind(Counterparty::getName,
Counterparty::setName);
                binder.forField(inn).bind(Counterparty::getInn,
Counterparty::setInn);
        binder.forField(address).bind(Counterparty::getAddress,
```

```
Counterparty::setAddress);
        binder.setBean(currentCounterparty);
        add(formLayout, saveButton, grid);
    }

    private void updateGrid() {
        grid.setItems(service.findAll());
    }

    private void clearForm() {
        currentCounterparty = new Counterparty();
        binder.setBean(currentCounterparty);
    }
}
```

2.3 Часть 3. Интегрируйте справочник в форму документа

Обновленное содержание класса `DocumentFormView.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.checkbox.Checkbox;
import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.datepicker.DatePicker;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.data.validator.RegexpValidator;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Counterparty;
import com.vrazhenko.documentfeature.CounterpartyService;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;

import java.time.LocalDate;

@Route(value = "document-form", layout = MainLayout.class)
public class DocumentFormView extends VerticalLayout {

    private final DocumentService documentService;
    private final CounterpartyService counterpartyService;
           private final Binder<Document> binder = new
Binder<>(Document.class);

            private final TextField titleField = new
TextField("Название");
          private final TextField regNumberField = new
```

```
TextField("Регистрационный номер");
          private final DatePicker regDatePicker = new
DatePicker("Дата регистрации");
        private final ComboBox<String> typeComboBox = new
ComboBox<>("Тип документа");

    private final ComboBox<Counterparty> counterpartyComboBox =
new ComboBox<>("Контрагент");

          private final Checkbox urgentCheckbox = new
Checkbox("Срочный документ");
    private final Button saveButton = new Button("Сохранить");
    private final Button clearButton = new Button("Очистить");

      public DocumentFormView(DocumentService documentService,
CounterpartyService counterpartyService) {
        this.documentService = documentService;
        this.counterpartyService = counterpartyService;

                typeComboBox.setItems("Приказ", "Распоряжение",
"Служебная записка");
        typeComboBox.setPlaceholder("Выберите тип");

         counterpartyComboBox.setItems(counterpartyService.findA
ll());
        counterpartyComboBox.setItemLabelGenerator(cp ->
                cp.getName() + " (ИНН: " + cp.getInn() + ")"
        );
        counterpartyComboBox.setRequired(true);

        FormLayout formLayout = new FormLayout();
                    formLayout.add(titleField, regNumberField,
regDatePicker,        typeComboBox,        counterpartyComboBox,
urgentCheckbox);

        formLayout.setResponsiveSteps(
                new FormLayout.ResponsiveStep("0", 1),
                new FormLayout.ResponsiveStep("500px", 2)
        );

        add(formLayout, saveButton, clearButton);

        binder.forField(titleField)
                .asRequired("Название не может быть пустым")
                .bind(Document::getTitle, Document::setTitle);

        binder.forField(regNumberField)
                .asRequired("Регистрационный номер обязателен")
                .withValidator(new RegexpValidator(
                          "Номер должен соответствовать формату
XXX-XX (например, 123-45)",
                        "^\\d{3}-\\d{2}$"))
                                    .bind(Document::getRegNumber,
```

```
Document::setRegNumber);

        binder.forField(regDatePicker)
                      .withValidator(date -> date != null && !
date.isAfter(LocalDate.now()),
                              "Дата регистрации не может быть позже
текущей")
                                          .bind(Document::getRegDate,
Document::setRegDate);

        binder.forField(typeComboBox)
                .asRequired("Выберите тип документа")
                .bind(Document::getType, Document::setType);

        binder.forField(counterpartyComboBox)
                                    .bind(Document::getCounterparty,
Document::setCounterparty);

        binder.forField(urgentCheckbox)
                .bind(Document::isUrgent, Document::setUrgent);

        saveButton.addClickListener(event -> saveDocument());
        clearButton.addClickListener(event -> clearForm());
    }

    private void saveDocument() {
        Document document = new Document();

        if (binder.writeBeanIfValid(document)) {
                    System.out.println("Сохранён документ: " +
document);

            documentService.saveDocument(document);

              Notification.show("Документ успешно сохранён: " +
document.getTitle());
            clearForm();
        } else {
              Notification.show("Пожалуйста, исправьте ошибки в
форме",
                     3000, Notification.Position.MIDDLE);
        }
    }

    private void clearForm() {
        binder.readBean(new Document());
    }
}
```

Обновленное содержание класса `DocumentListView.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.grid.GridSortOrder;
import com.vaadin.flow.component.orderedlayout.FlexComponent;
import
com.vaadin.flow.component.orderedlayout.HorizontalLayout;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.provider.SortDirection;
import com.vaadin.flow.data.value.ValueChangeMode;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;

@Route(value = "documents", layout = MainLayout.class)
public class DocumentListView extends VerticalLayout {

    private final DocumentService documentService;

            private   final   Grid<Document>   grid   =   new
Grid<>(Document.class);
     private final TextField searchField = new TextField("Поиск
по названию");
         private   final   ComboBox<String>   typeFilter   =   new
ComboBox<>("Тип документа");

          public   DocumentListView(@Autowired   DocumentService
documentService) {
        this.documentService = documentService;

                      searchField.setPlaceholder("Введите   часть
названия...");
        searchField.setClearButtonVisible(true);
        searchField.setValueChangeMode(ValueChangeMode.LAZY);
        searchField.addValueChangeListener(e -> updateList());

           typeFilter.setItems("Все", "Приказ", "Распоряжение",
"Служебная записка");
        typeFilter.setValue("Все");
        typeFilter.setClearButtonVisible(false);
        typeFilter.addValueChangeListener(e -> updateList());

                           HorizontalLayout   filters   =   new
HorizontalLayout(searchField, typeFilter);
        filters.setAlignItems(FlexComponent.Alignment.END);

        configureGrid();
```

```
        add(filters, grid);

        updateList();
    }

    private void configureGrid() {
        grid.removeAllColumns();

        grid.addColumn(Document::getTitle)
                .setHeader("Название")
                .setSortable(true)
                .setKey("title");

        grid.addColumn(Document::getRegNumber)
                .setHeader("Регистрационный номер");

        grid.addColumn(Document::getRegDate)
                .setHeader("Дата регистрации")
                .setSortable(true)
                .setKey("regDate");

        grid.addColumn(Document::getType)
                .setHeader("Тип документа");

        grid.addColumn(Document::isUrgent)
                .setHeader("Срочный документ")
                .setKey("urgent");

        grid.setSizeFull();

        grid.sort(List.of(
                                                                new
GridSortOrder<>(grid.getColumnByKey("regDate"),
SortDirection.DESCENDING)
        ));

          grid.addColumn(doc -> doc.getCounterparty() != null ?
doc.getCounterparty().getName() : "")
                .setHeader("Контрагент")
                .setKey("counterparty");
    }

    private void updateList() {
        String searchTerm = searchField.getValue();
        String selectedType = typeFilter.getValue();
         grid.setItems(documentService.findDocuments(searchTerm,
selectedType));
    }
}
```

Результат добавления выпадающего списка приведен на Рисунке 2.

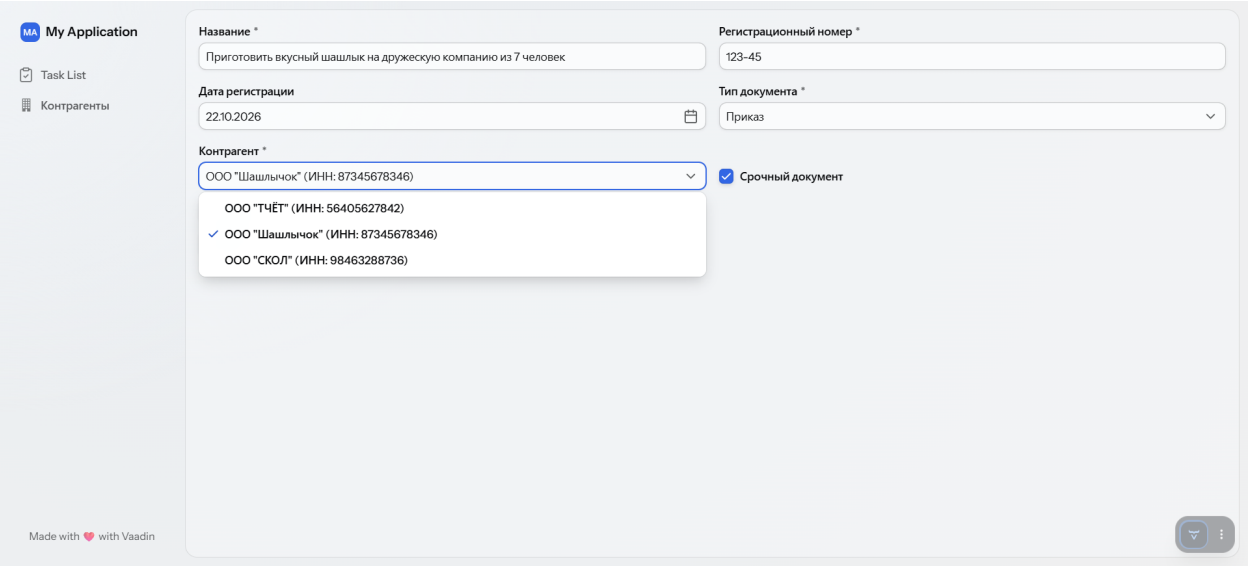


Рисунок 2 – Результат добавления выпадающего списка

2.4 Часть 4. Стилизация и кастомизация (UI Tuning)

Обновленное содержание класса `DocumentFormView.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.button.ButtonVariant;
import com.vaadin.flow.component.checkbox.Checkbox;
import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.combobox.ComboBoxVariant;
import com.vaadin.flow.component.datepicker.DatePicker;
import com.vaadin.flow.component.datepicker.DatePickerVariant;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.component.textfield.TextFieldVariant;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.data.validator.RegexpValidator;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Counterparty;
import com.vrazhenko.documentfeature.CounterpartyService;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;

import java.time.LocalDate;

@Route(value = "document-form", layout = MainLayout.class)
public class DocumentFormView extends VerticalLayout {
```

```
    private final DocumentService documentService;
    private final CounterpartyService counterpartyService;
            private final Binder<Document> binder = new
Binder<>(Document.class);

             private final TextField titleField = new
TextField("Название");
           private final TextField regNumberField = new
TextField("Регистрационный номер");
           private final DatePicker regDatePicker = new
DatePicker("Дата регистрации");
         private final ComboBox<String> typeComboBox = new
ComboBox<>("Тип документа");
    private final ComboBox<Counterparty> counterpartyComboBox =
new ComboBox<>("Контрагент");
           private final Checkbox urgentCheckbox = new
Checkbox("Срочный документ");

    private final Button saveButton = new Button("Сохранить");
    private final Button clearButton = new Button("Очистить");

      public DocumentFormView(DocumentService documentService,
CounterpartyService counterpartyService) {
        this.documentService = documentService;
        this.counterpartyService = counterpartyService;

        titleField.addThemeVariants(TextFieldVariant.LUMO_SMALL
);
        regNumberField.addThemeVariants(TextFieldVariant.LUMO_S
MALL);
        regDatePicker.addThemeVariants(DatePickerVariant.LUMO_S
MALL);
        typeComboBox.addThemeVariants(ComboBoxVariant.LUMO_SMAL
L);
        counterpartyComboBox.addThemeVariants(ComboBoxVariant.L
UMO_SMALL);

        saveButton.addThemeVariants(ButtonVariant.LUMO_PRIMARY)
;
        clearButton.addThemeVariants(ButtonVariant.LUMO_TERTIAR
Y);

                typeComboBox.setItems("Приказ", "Распоряжение",
"Служебная записка");
        typeComboBox.setPlaceholder("Выберите тип");

        counterpartyComboBox.setItems(counterpartyService.findA
ll());
               counterpartyComboBox.setItemLabelGenerator(cp ->
cp.getName() + " (ИНН: " + cp.getInn() + ")");
        counterpartyComboBox.setRequired(true);
```

```
        FormLayout formLayout = new FormLayout();
                    formLayout.add(titleField,  regNumberField,
regDatePicker,        typeComboBox,        counterpartyComboBox,
urgentCheckbox);
        formLayout.setResponsiveSteps(
                new FormLayout.ResponsiveStep("0", 1),
                new FormLayout.ResponsiveStep("500px", 2)
        );

        add(formLayout, saveButton, clearButton);

        binder.forField(titleField)
                .asRequired("Название не может быть пустым")
                .bind(Document::getTitle, Document::setTitle);

        binder.forField(regNumberField)
                .asRequired("Регистрационный номер обязателен")
                .withValidator(new RegexpValidator(
                        "Номер должен соответствовать формату
XXX-XX (например, 123-45)",
                       "^\\d{3}-\\d{2}$"))
                                    .bind(Document::getRegNumber,
Document::setRegNumber);

        binder.forField(regDatePicker)
                      .withValidator(date -> date != null &&  !
date.isAfter(LocalDate.now()),
                        "Дата регистрации не может быть позже
текущей")
                                       .bind(Document::getRegDate,
Document::setRegDate);

        binder.forField(typeComboBox)
                .asRequired("Выберите тип документа")
                .bind(Document::getType, Document::setType);

        binder.forField(counterpartyComboBox)
                                  .bind(Document::getCounterparty,
Document::setCounterparty);

        binder.forField(urgentCheckbox)
                .bind(Document::isUrgent, Document::setUrgent);

        saveButton.addClickListener(event -> saveDocument());
        clearButton.addClickListener(event -> clearForm());
    }

    private void saveDocument() {
        Document document = new Document();

        if (binder.writeBeanIfValid(document)) {
                    System.out.println("Сохранён документ: " +
document);
```

```
            documentService.saveDocument(document);

              Notification.show("Документ успешно сохранён: " +
document.getTitle());
            clearForm();
        } else {
              Notification.show("Пожалуйста, исправьте ошибки в
форме", 3000, Notification.Position.MIDDLE);
        }
    }

    private void clearForm() {
        binder.readBean(new Document());
    }
}
```

Обновленное содержание класса DocumentListView.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.dependency.CssImport;
import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.grid.GridSortOrder;
import com.vaadin.flow.component.orderedlayout.FlexComponent;
import
com.vaadin.flow.component.orderedlayout.HorizontalLayout;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.provider.SortDirection;
import com.vaadin.flow.data.value.ValueChangeMode;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;

@CssImport("./styles/grid-styles.css")
@Route(value = "documents", layout = MainLayout.class)
public class DocumentListView extends VerticalLayout {

    private final DocumentService documentService;

            private final Grid<Document> grid = new
Grid<>(Document.class);
     private final TextField searchField = new TextField("Поиск
по названию");
         private final ComboBox<String> typeFilter = new
```

```
ComboBox<>("Тип документа");

          public   DocumentListView(@Autowired   DocumentService
documentService) {
        this.documentService = documentService;

                      searchField.setPlaceholder("Введите   часть
названия...");
        searchField.setClearButtonVisible(true);
        searchField.setValueChangeMode(ValueChangeMode.LAZY);
        searchField.addValueChangeListener(e -> updateList());

           typeFilter.setItems("Все", "Приказ", "Распоряжение",
"Служебная записка");
        typeFilter.setValue("Все");
        typeFilter.setClearButtonVisible(false);
        typeFilter.addValueChangeListener(e -> updateList());

                            HorizontalLayout   filters   =   new
HorizontalLayout(searchField, typeFilter);
        filters.setAlignItems(FlexComponent.Alignment.END);

        configureGrid();

        add(filters, grid);

        updateList();
    }

    private void configureGrid() {
        grid.removeAllColumns();

        grid.addColumn(Document::getTitle)
                .setHeader("Название")
                .setSortable(true)
                .setKey("title");

        grid.addColumn(Document::getRegNumber)
                .setHeader("Регистрационный номер");

        grid.addColumn(Document::getRegDate)
                .setHeader("Дата регистрации")
                .setSortable(true)
                .setKey("regDate");

        grid.addColumn(Document::getType)
                .setHeader("Тип документа");

        grid.addColumn(Document::isUrgent)
                .setHeader("Срочный документ")
                .setKey("urgent");

         grid.addColumn(doc -> doc.getCounterparty() != null ?
```

```
doc.getCounterparty().getName() : "")
                .setHeader("Контрагент")
                .setKey("counterparty");

        grid.setSizeFull();

        grid.sort(List.of(
                                                                new
GridSortOrder<>(grid.getColumnByKey("regDate"),
SortDirection.DESCENDING)
        ));
    }

    private void updateList() {
        String searchTerm = searchField.getValue();
        String selectedType = typeFilter.getValue();
         grid.setItems(documentService.findDocuments(searchTerm,
selectedType));
    }
}
```

Содержание файла со стилями `grid-styles.css` представлено в листинге ниже:

```
vaadin-grid::part(header-cell) {
    background-color: #e8f5e9;
    color: #2e7d32;
    font-weight: bold;
}
```

Результат применения стилей представлен на Рисунке 3.

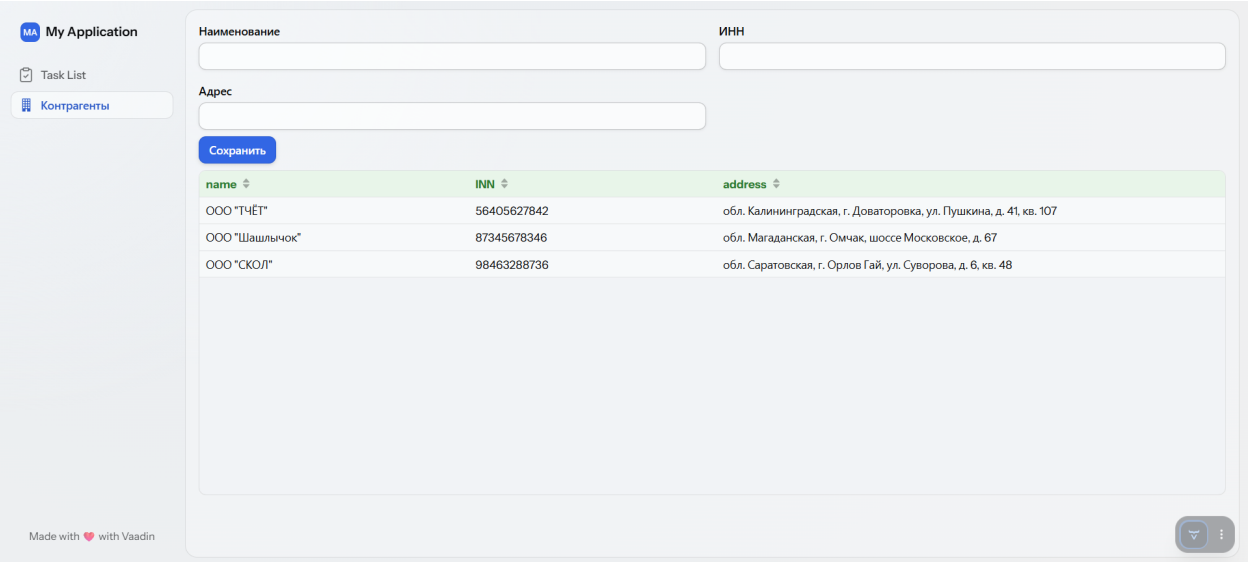


Рисунок 3 – Результат применения стилей

3. Вывод

В ходе практической работы был создан справочник «Контрагенты» и настроена связь «многие-к-одному» (`@ManyToOne`) с сущностью документа. Использование внешнего ключа обеспечило ссылочную целостность данных и устранило дублирование информации.

Для управления справочником реализован CRUD-интерфейс. Интеграция `ComboBox` в форму документа (с отображением названия и ИНН) и связывание через `Binder` упростили выбор контрагента и исключили ошибки ручного ввода.

Также выполнена кастомизация интерфейса: применены встроенные темы Vaadin (`LUMO_PRIMARY`, `LUMO_TERTIARY`, `LUMO_SMALL`) для кнопок и полей ввода, а с помощью внешнего CSS-файла изменен стиль заголовков таблицы `Grid`. Полученные навыки являются базовыми для разработки корпоративных систем документооборота.

4. Ответы на контрольные вопросы

1. В чем преимущество хранения ссылок на справочники через ID (Foreign Key) вместо хранения текстовых названий прямо в таблице документов?

Обеспечивается целостность данных и устраняется дублирование. При изменении названия контрагента достаточно обновить одну запись в справочнике, а не править тысячи документов.

2. Какая аннотация в Vaadin позволяет подключить внешний CSS-файл к конкретному View?

Аннотация `@CssImport` (например, `@CssImport("./styles/style.css")`).

3. Как ограничить высоту выпадающего списка в ComboBox, если в справочнике слишком много записей?

Через CSS для оверлея компонента или программно с помощью метода `setMaxHeight("300px")`. Для больших объемов данных лучше использовать ленивую загрузку.

4. Чем FetchType.LAZY отличается от FetchType.EAGER?

`EAGER` загружает связанную сущность сразу вместе с основным объектом. `LAZY` откладывает загрузку до момента первого обращения к связанному полю (экономит ресурсы).

5. Что такое нормализация базы данных?

Это процесс организации данных в таблицах для устранения избыточности и предотвращения аномалий при изменении данных.

6. Назовите три основные цели нормализации.

- Устранение дублирования данных.
- Предотвращение аномалий (ошибок при обновлении/удалении).
- Обеспечение логической целостности.

7. Какие три аномалии устраняет нормализация?

Аномалия вставки (Insert), аномалия обновления (Update) и аномалия удаления (Delete).

8. Сколько нормальных форм существует? Перечислите их по порядку.

Шесть: 1НФ, 2НФ, 3НФ, НФБК (BCNF), 4НФ, 5НФ.

9. Какие виды связей существуют в JPA?

`@OneToOne` (один-к-одному), `@OneToMany` (один-ко-многим), `@ManyToOne` (многие-к-одному), `@ManyToMany` (многие-ко-многим).

10. Зачем нужен атрибут mappedBy?

Он указывает на «ведомую» сторону в двунаправленной связи, сообщая Hibernate, что управление внешним ключом происходит через другое поле (в другом классе).

11. Опишите словами, как в базе данных реализуется связь Many-toMany. Какие таблицы появляются?

Создается третья (промежуточная/связующая) таблица, которая содержит внешние ключи (ID) от двух связываемых таблиц.

12. Что произойдет, если в справочнике будет 1 000 000 записей, а ComboBox загрузит их все?

Приложение столкнется с переполнением памяти (`OutOfMemoryError`), браузер зависнет, а сеть будет перегружена передачей огромного JSON. Необходима ленивая загрузка (Lazy Loading) через `DataProvider`.